Rapid integration and testing of business solutions

C J Carter, I G Whyte, S A Birchall and D Swatman

*Rapid application development (RAD) methodology is increasingly being used by software development units across BT as a means of delivering maximum functionality in the shortest time thereby allowing the company to respond quickly to new business opportunities. DSDM (Dynamic Systems Development Method) has emerged as the UK and international standard for RAD and is currently being used by a number of development units within BT. For a number of reasons, some of the fundamental tenets of DSDM are proving difficult to use for small to medium size multimedia and business solution projects. This paper looks at how the Advanced Service Management Unit of BT's Systems Integration Department has responded to this situation and formulated a hybrid approach to testing known as '**SI-Rapid**'.*

1. Introduction

There is an ever-increasing demand within business for customer solutions to be developed and delivered in shorter time-scales, at lower cost, while maintaining high standards of quality. These apparently conflicting requirements are not easily met by software developments based on the traditional 'waterfall' life cycle model [1, 2] that is based on the serial operation of processes with consequent lengthy time delay between inception and delivery. In recent years there has been much interest in adopting 'rapid application development' (RAD) methodologies — especially for small-scale applications — because they encourage much parallel activity across the development life cycle and hence offer earlier delivery.

One RAD methodology, DSDM — produced by the Dynamic Systems Development Methodology Consortium of which BT is a member — has emerged as a UK and international standard for rapid software development. However, a number of the key elements of DSDM, e.g. one room working, are proving difficult to implement for anything other than very small projects, as BT has a geographically disparate customer/development/test/installation team structure. As a consequence, a form of 'half-way house' working has evolved with some parts of the traditional life cycle approach being used in conjunction with parts of DSDM.

This paper outlines the principle of DSDM as seen from the systems integration and test viewpoint and then looks at how the best elements of the more traditional approach can be combined with the DSDM ideal to form a pragmatic method based on practical experience — ' **SI-Rapid**'.

Solutions such as those proposed by the DSDM Consortium and practically implemented by means of ' **SI-Rapid**' require considerable discipline and a degree of culture change for all members of the project team. Without such a change, customer demands for cost reductions and shortened delivery time-scales — especially for niche market products and services — will remain unfulfilled.

2. The millennium challenge

As we move towards the next millennium, it is clear that the pressure to deliver rapid business solutions will continue to increase. Systems integration and development teams will be faced with many new challenges as time, cost and quality factors are juggled continuously. Quality must be maintained yet many of the underpinnings of quality which are part and parcel of the traditional 'waterfall' life cycle approach may not be there for support, because, in the rush to deliver, documentation and those activities associated with 'slowing the process down' tend to be brushed aside. The following sections of the paper examine the traditional 'waterfall' life cycle approach, DSDM, and conclude with a proposal for making ' **SI-Rapid**' work in practice. Additional information can be found elsewhere [1, 3, 4].

2.1 Traditional integration and test

A fundamental tenet of the systems integration (SI) process (as defined in BT's documentation [1—3]) is that it should be bounded by an incoming quality gate (which must be passed before testing can commence) and an outgoing

quality gate (which must be passed before deployment or customer acceptance can begin). Also important is that the software to be tested should be delivered by way of a configuration management system so that there is no misunderstanding of the deliverable content.

During the testing period, faults are recorded using problem reports (PRs) which then follow a prescribed resolution process as shown in Fig 1. A full description of the current process based on the traditional 'V' model methodology is described in the BT VV&T Handbook [1].

The quality gates referred to above provide a mechanism whereby the test team can assess a delivery against an agreed and documented set of conditions. Typically an input quality gate requires that software is delivered with a release note stating known defects and problems, the level of unit testing performed, results of static code analysis (e.g. QA for 'C') and memory leakage (e.g. Purify). There should also be fully documented installation instructions from which the software can be installed and run.

Once into integration and testing, work proceeds as stated in the project verification, validation and testing (VV&T) strategy. This is supported by a series of test specifications — which define how testing will be performed, for example, stress, load and performance — and detailed test cases which state precisely the start conditions, the test to be performed and the expected result. Faults and defects will be found during testing, these must be documented and fed back to the development team for fixing. Some degree of customer involvement in the prioritisation of fault fixing may be necessary, as some faults are deviations from requirement that may be acceptable or of low priority.

2.2 The waterfall model

Strengths

The traditional waterfall model for systems integration and test is well understood, established and proven. All processes are fully documented (for examples, see the BT VV&T Handbook [1] and the Systems Integration Handbook [2]) and hence provide comprehensive support for downstream maintainability and the quality management system.

The waterfall process also allows (via extensive documentation) full traceability to take place from requirements testability analysis at the beginning of the project through to the outgoing product test summary report.

Weaknesses as seen from a RAD perspective

Waterfall life cycle model developments can be very time consuming and hence time-scales tend to be lengthy. Yet not all of this time can be used to best effect because the input documentation to systems integration, e.g. requirements/compliance statement documentation, is often uncontrolled and subject to continual refinement. As all subsequent test documentation — from strategy through to discrete test case — is dependent on this unstable source documentation, early test design and test cases preparation suffers and may need to be continually reviewed and updated.

Due to the separate identity of development and test teams, walls can be built which inhibit co-operation and promote the 'us and them' attitude especially when there is time pressure. Informal testing by the test teams during development can also lead to difficulties as testing is seen as being a delaying influence.

The software and associated documentation can take many attempts and a long time to pass the input quality gates. As a consequence feedback from test to development can come almost too late as the development team will have started to work on the next phase or release and their mind-set will have changed.

3. DSDM — meeting the challenge

One of the ways in which BT has been rising to the challenge presented by the traditional 'waterfall' approach to integration and test is through the adoption of the RAD philosophy for certain types of project. The only formalised method of RAD currently in use is DSDM (Dynamic Systems Development Method) [4]. DSDM is a non-proprietary rapid application development method produced by a consortium, of which BT is a member, comprising a number of vendors, users and associates. BT has chosen to adopt DSDM as it has become established as the UK and international standard for RAD.

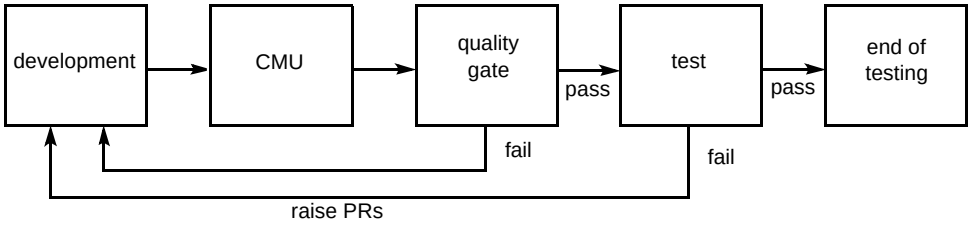

Fig 1 Typical delivery/testing process.

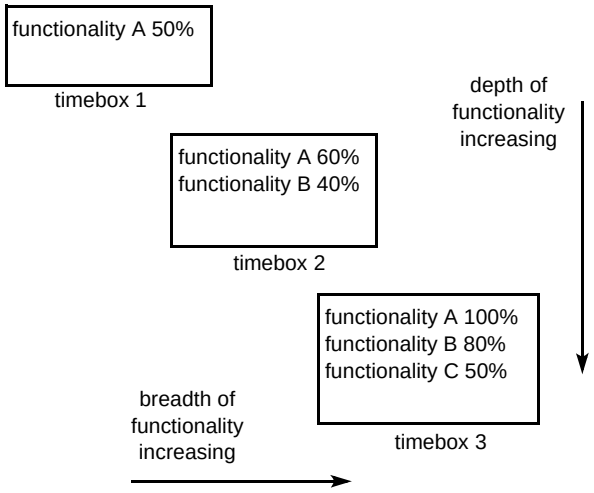

Fig 2 Iterative prototyping.

Fundamental to DSDM is the iterative manner in which prototypes are initially evolved and then built upon to form the final customer deliverable. Figure 2 shows how the 'prototyping spiral' provides increased functionality in both breadth and depth as the system evolves towards the intended final solution.

DSDM is a 'development life cycle', with associated techniques that aim to:

- reduce development time-scales by actively minimising the time taken to go from initial customer requirements to delivered application,
- contain and reduce development costs by tightly controlling timeboxing,
- focus on meeting the customer requirements,
- ensure that testing is a continuous process right from the beginning.

DSDM depends on:

- user (customer) involvement throughout,
- devolving decisions to the team,
- workshops,
- prototyping,
- timeboxing,
- co-location of customer, development and test teams,
- appropriate use of tools (e.g. CASE tools).

3.1 DSDM testing principles

There are five key DSDM testing principles, as listed below:

- validation testing — the primary goal of testing is to make sure that a system meets the real business requirements,
- benefit-directed testing — testing should be focused on those parts of a system that deliver primary business benefits,
- testing throughout the life cycle — testing is performed at all stages of development and will begin once the first module has been produced,
- independent testing — testing should be performed by someone other than the author,
- repeatable testing — all tests must be repeatable.

By virtue of the iterative nature of the DSDM process, much testing is done on prototypes so as to get the earliest possible visibility of software performance. Prototype testing has three distinctly different roles:

- to illustrate a partial build of the system,
- as a technique for gathering information,
- to clarify functional or technical requirements.

Different test criteria apply to each timebox as the product matures and depends heavily on the role the prototype is serving.

- If a prototype offers part functionality of the final delivered system then test pass/fail criteria can be matched to **expected results** based on known requirements.
- If the prototype is intended only to clarify requirements, the criteria for test success/failure might be whether or not the prototype generates the **expected information**. The contrast here is that an expected result can be predefined whereas expected information cannot. Hence, in this case, the prototype is being used to expand the knowledge and understanding of the prototype builder and will be considered successful if this is achieved.

3.2 DSDM testing techniques and tools

Due to the short development time-scales imposed by timeboxing, testing tends to be informal. Test documentation is minimal and often contains only a list of conditions to test for in order to identify test coverage and provide an indication of prioritisation. Tracking of tests passed or failed is made using this same list. The likely result of a test will be left to the tester to determine and will not be documented.

In order to maximise benefit given time constraints, testing is based around end-user scenarios, i.e. testing works

inwards from the user interface. It is left to the developers to test non-visible functionality, such as database updates or non-functional test aspects, such as stress, load and performance.

DSDM recommends the use of static code analysers (e.g. QA for 'C') since tools of this type can provide a quick and easy way of gaining feedback without having to wait for input from other parties. Due to the likely lack of formal test designs, for reasons discussed earlier, dynamic analysis tools, such as array boundary checkers and memory leakage detectors, are strongly recommended, as are capture and replay tools. These, however, are only of real benefit if the user interface can be stabilised early in the project so that automated test scripts do not need a high level of maintenance. However, once user-interface stability has been reached, automated tests can be run nightly as the product is developed to give high levels of regression confidence.

3.3 Strengths/weaknesses

Strengths

In applying the principle of the 80:20 rule, DSDM recognises that 80% of the functionality is produced in 20% of the time. Further, it assumes that the application will not be built correctly first time but will improve with iteration. However, unlike the waterfall methodology where a step-wise approach is taken to development, i.e. complete step (a) and then move on to step (b), DSDM requires that all previous steps be re-appraised at each cycle.

The timeboxing approach of DSDM offers an iterative approach to requirements capture. Typically, many man-months of effort are applied to requirements capture and at the end of the activity the customer can often still be unclear of what he wants. Iterative prototyping addresses this issue.

The process does not have to be strong and strictly regimented.

There is a primary focus on business requirements rather than development need. The 80:20 rule ensures that requirements are prioritised so that those that provide the greatest business benefit are delivered first. Non-essential requirements are dealt with in subsequent releases, if still considered appropriate.

Weaknesses

DSDM is often seen as a license to hack (i.e. work can be done without any formal process or project management control.) However, adoption of the DSDM methodology does not remove the necessity for close control. Furthermore, it is suggested that a much higher standard of project management is required in order to keep the project focused. Quality remains an issue because there is no strict ISO quality framework to work from. This can present problems when other parts of the organisation are required to maintain strict quality standards. These and other problems are well recognised by industry, the most important of which are discussed below.

Containment of scope (requirements)

Due to the lax way that requirements tend to be managed, the scope of a project can easily wander as new requirements are thrown up during development or at the end of a timebox. Fixed duration timeboxes can help minimise this undesirable tendency but it must be accepted that changes will be frequent and inconvenient.

Positive management of requirements in conjunction with customer sign-off is important; as is the need to track new and existing requirements regularly, e.g. at customer review meetings and decide to which timebox they will apply.

Managing alterations

Active management of change requires rigorous use of configuration management (CM).

Lack of established guidelines

Unlike DSDM, most RAD-type methodologies do not have a defined development process. However, even DSDM only provides limited guidance on testing [4].

Lack of testing with DSDM developed products

RAD developed products are reported in the industry generally to suffer from lack of testing and that even when testing has been performed, few faults have been found. Testing is often carried out by customers and end-users who have negligible integration or testing experience and as such do not have the experienced tester's mind-set. Within DSDM, the benefit of independent testing tends to be under-emphasised.

Creeping functionality

Creeping functionality is a problem in any development, but is particularly tempting in a DSDM one where numerous timeboxes provide numerous opportunities. Strong project and customer management is required to ensure that 'frills' and unjustified extra functionality are kept at bay.

Lack of customer involvement

Due to the need to gain agreement rapidly on the interpretation of requirements and proposed solutions, it is

important to have the customer fully integrated with the project team. However, customers find this difficult to achieve in practice and alternative communication methods, e.g. audio-/video-conferencing, may be necessary to ensure that regular close contact is maintained.

Documentation

As time is at an absolute premium within the DSDM timebox, production of even a minimum 'documentation set' can prove difficult to achieve.

4. Optimising the solution — SI Rapid

For a number of reasons, the use of DSDM in its pure form is proving difficult within BT. Geographically disparate customers, development, and systems integration teams present a particular challenge, as one of the fundamental DSDM concepts — that of 'one-room working'— cannot be applied to most projects. Additionally it has been noted that many customers request that their project be done by RAD because they want fast delivery for obvious reasons. This can be difficult to reconcile because most BT projects involve significant integration with large legacy systems and require certain conditions to be met before 'approval to connect' can be granted. This can be difficult to obtain in a RAD environment. However, as demand for rapid business system development continues to grow, there is a genuine need for a pragmatic solution to be found. The answer does not lie in the traditional approach either. BT Systems Integration (SI) has been moving towards a half-way house approach for certain types of projects in an attempt to provide a combination of rapid integration and test within a quality-based maintainable framework; this process is called ' **SI-Rapid**'.

While it is recognised that Systems Integration will normally be asked to input their view at project conception, the decision to use rapid or other development methodologies is largely down to customer preference and the need to hit a given delivery milestone. However, as Systems Integration are key members of the overall delivery process it is important for them to be involved from the earliest possible opportunity and to have their input noted when reviewing and agreeing the development approach to be taken. Should the decision to adopt a RAD methodology be made, the **SI-Rapid** process is intended to provide an effective integration and test method within the time and budget constraints of the project.

Early SI presence, i.e. at the requirements capture workshop, should be a mandatory precondition for undertaking work. As a second option, the availability of either fully documented requirements or a compliance statement may be acceptable. A large part of the work performed by Systems Integration in the early days of the project — the V&V stage — provides important information about the risks involved in taking the RAD approach, the nature and scope of testing and test prioritisation. This initial phase provides the foundation for all future work and will also include agreement on the location and type of the test model(s), configuration management, software delivery and fault-reporting methods and tools.

4.1 SI strategy and risk management

Ideally, an analysis of the statement of requirements or compliance statement will provide input to the project by means of a requirements testability analysis document. Requirements that cannot be tested or are ambiguous must either be rewritten at this stage or be quantified by means of a risk statement. It is arguable as to whether requirements which are vague or untestable can be built into the design and subsequently coded, yet it is often the test engineer — brought in late in the product life cycle — who raises the issue. A full analysis of the risks associated with taking the RAD approach for a project is considered essential. The test strategy adopted should link closely to the risk analysis because there are many different types of test scenario possible. Four principal test categories can be identified:

- full, comprehensive, end-to-end system testing (which is unlikely to be appropriate in a RAD environment due to time constraints and the lack of a complete system until the final timebox),
- 80:20 rule functionality prioritised testing where testing comprises of a mixture of functional testing, non-functional testing and customer user scenario testing — this option requires that the risk of having certain areas of functionality minimally tested is understood and agreed,
- rapid testing which focuses on defined work processes or user scenarios (this is known as scenario-based testing) — in this case only the top 20% of the requirements covering 80% of the system functionality would be tested and no time would be spent on the non-functional aspects (stress, load and performance, for example),
- finally, there is the option of 'user testing with SI consultancy' for those projects where time to deployment is of the essence and the risks are clearly understood.

In practice, the nature of testing changes as the project evolves. The last option listed above, 'user testing with SI consultancy', is not to be recommended for anything other than simple system upgrades, e.g. the change from a VT100 'dumb' terminal to a PC-based graphical user interface. When the timeboxing approach to RAD is taken, the first timebox test activity aims at finding those areas of weakness or deficiencies that have critical impact. Under

these circumstances, rapid testing is the most appropriate choice as it is likely that major changes will occur in later timebox releases. Once product stability increases, especially at the user interface, the 80:20 rule-based testing regime is more appropriate as at this stage it will be important to gain an understanding of how the system copes with loading as well as incorrect user actions. It is unlikely that full end-to-end testing will be performed in a RAD development, but it can be expected that, by the final timebox, testing will have gone beyond the 80:20 rule level in many areas.

4.2 Development timebox

DSDM promotes strongly the use of iterative prototyping within timeboxes as shown previously in Fig 2.

Figure 3 shows diagrammatically in more detail the activities that take place during one phase of the development cycle.

Within the development timebox of Fig 3, there are more detailed activities shared between development and test as shown in Fig 4. Note that some of the activities are the responsibility of the test team (shown shaded), while others are owned by development but have an active SI involvement. The activities shown in the development timebox are discussed below in more detail.

A testing activity not shown on Fig 4 is that of user scenario production which starts once the requirements are defined and continues to be refined during each development iteration within the timebox. Detailed test cases are developed from these user scenarios as illustrated later in Fig 7.

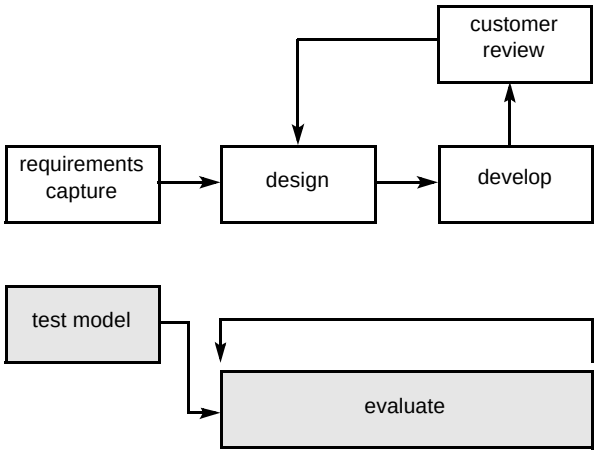

Fig 4 Development timebox.

Requirements capture

It is important that the test engineers have sufficient understanding of the project requirements to form a basis for testing. Frequently the test team will need to seek clarification or rewrite requirements as DSDM-style projects tend to be loose in this respect. Any such evolution must be agreed with both customer and development teams in order to ensure a common understanding.

Time should be made for verification, validation and general requirement analysis to be carried out by the test team. It is recognised that time pressure may be used as an excuse for not doing this; however, effort must be put into ensuring that requirements are testable, and can be designed and coded.

Inadequate requirements

There are an increasing number of occasions when the requirements are so limited that the test team is faced with the dilemma of having to test the software against itself. In this instance, testers will be faced with the situation of observing system behaviour, documenting test cases ‘on the

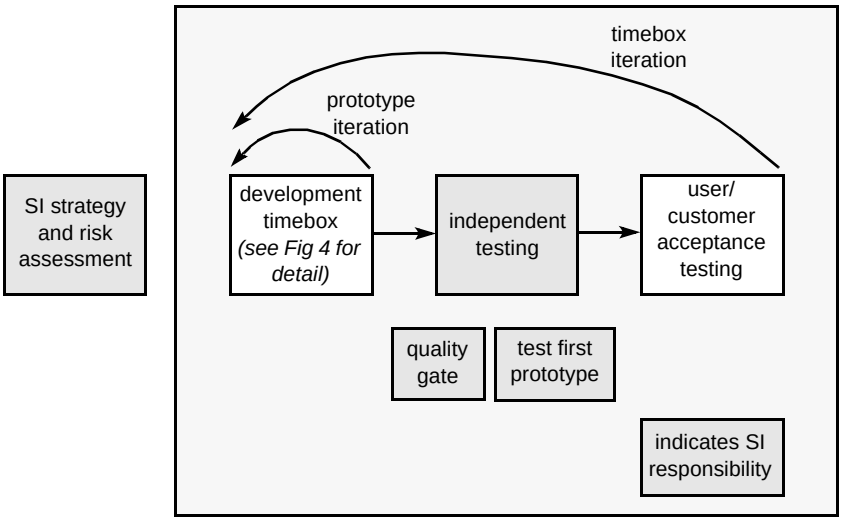

Fig 3 ‘**SI-Rapid**’ life cycle.

fly' then having to decide if that behaviour is correct. This is clearly unsatisfactory and will severely limit the scope of testing and greatly increase the risk that many problems will only be detected once the software has been deployed.

Design

Testers must work alongside the systems designer right from the beginning. By this means, the test team can offer VV&T input to the design process as well as gain valuable insight of likely deployment configurations.

Develop

Following design, developers start production of the first prototype. Figure 5 shows how, using the iterative process, a prototype is developed, used and evaluated by systems integration before being passed back for more iteration round the life cycle loop.

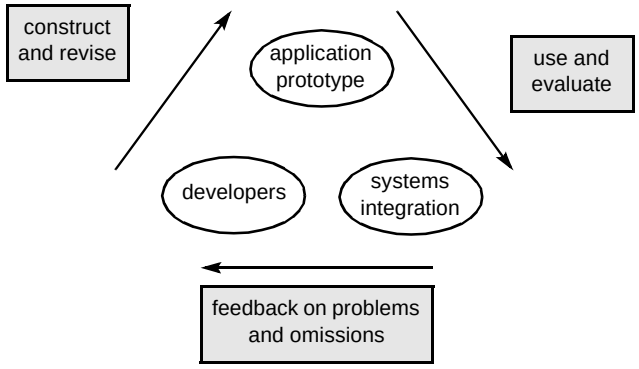

Fig 5 Prototyping iteration.

Test model

The test team begins to configure a test environment either as a separate systems integration and test facility or co-located within the development 'clean room'.

Evaluate

The evaluation sub-phase is performed by the test team working closely with development once some usable parts of the software system have been coded. This may take place on the development machine and can be a very useful stage for harmonising the user-interface presentation across several developers.

As software production becomes more advanced, user scenarios will be drafted against process workstrings and procedures in conjunction with elements of the customer's requirements. This scenario-based method of testing can be very effective in determining how well major areas of functionality perform and provides valuable early feedback.

Evaluation results in the following:

- timely feedback on the behaviour of the software is available in a constructive non-threatening way — at this stage (and particularly so on the first iteration), team building between 'development' and 'test' is important and the use of formal fault-reporting methods, e.g. PCMS, may be inappropriate,

- the test team is able to produce draft test cases from which they will then be able to refine more detailed test cases as well as providing an indication of which areas are likely to need more extensive testing later.

Evaluation may take place alongside developers or even on a development machine if appropriate. (It should be noted that this is an expedient at this stage as the development team is unlikely to be able to provide a separate build for testing. However, in the longer term, the use of an independent test machine representative of the deployed system configuration must be used because the development environment is often poorly configuration managed and may have spurious files present on which the delivery is unwittingly dependent.)

This evaluation of software deliveries by the test team will continue through development until the software is declared complete and believed to be ready for the test input quality gate. At this time it will be necessary for the project to move towards a more formal approach to problem reporting, e.g. the use of PCMS. Once the quality gate has been passed, the test team can begin testing in earnest using test cases based on any available requirements, designs and practical knowledge of the system.

As the product matures through the second or third timebox, informal testing becomes less appropriate and testing will take on a more prioritised focus using progressively refined test cases.

In summary, the testing described here builds steadily from informal observation and comment on incomplete modules through to properly managed tests (possibly under the control of a test management tool). Formal fault reporting will be used to support full configuration management.

Customer review

It is important that a member of the test team attends the developer/customer product review meetings as they present an opportunity to gather first hand knowledge of the customer's initial reaction. By this time the test team may well have a better understanding of how, in overall terms, the product works and provide significant input in their own right. Customer feedback can be noted — screen dumps can provide a simple, quick and easy route — so that the test

team can take any comments into account when refining their test documentation set.

This represents the last stage in the activity as shown in Fig 4. Depending upon how the review went, it may be necessary to cycle round within the timebox more than once. At least two customer reviews should be scheduled in each timebox so that any deviation from plan is rapidly dealt with.

4.3 Independent testing timebox

The independent testing phase of the ‘ **SI-Rapid**’ process is shown in Fig 6, and the full process in Fig 7.

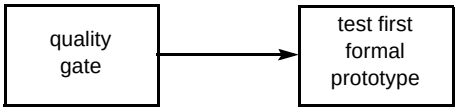

Fig 6 Independent testing.

A set of deliverables (as a configuration managed, documented build) is subject to a formal quality gate. The success rate of this quality gate should in theory be very high because the independent test team will have been working closely with the developers throughout the development process.

Quality gate

Quality gates should not strictly be necessary when working in a DSDM-style testing environment. However, practice shows that development communities often want to develop in their own style and that what they define as a deliverable build may not meet systems integration expectations. For this reason it is recommended that the quality gate process is retained to ensure that a documentation set, however minimal, is produced and that a clearly defined end to a timebox is apparent.

Pryke et al [5] provide more information on the application of quality gates.

Test first formal prototype

Once the first formal prototype has passed its quality gate, independent testing can begin. The test team will undertake testing on an independent, configuration managed, test model representative of the deployed system in terms of completeness and complexity. Any outstanding observations will be discussed with the development team at the start of this activity and raised as formal problem reports. At the end of the testing activity, a test summary report will be written.

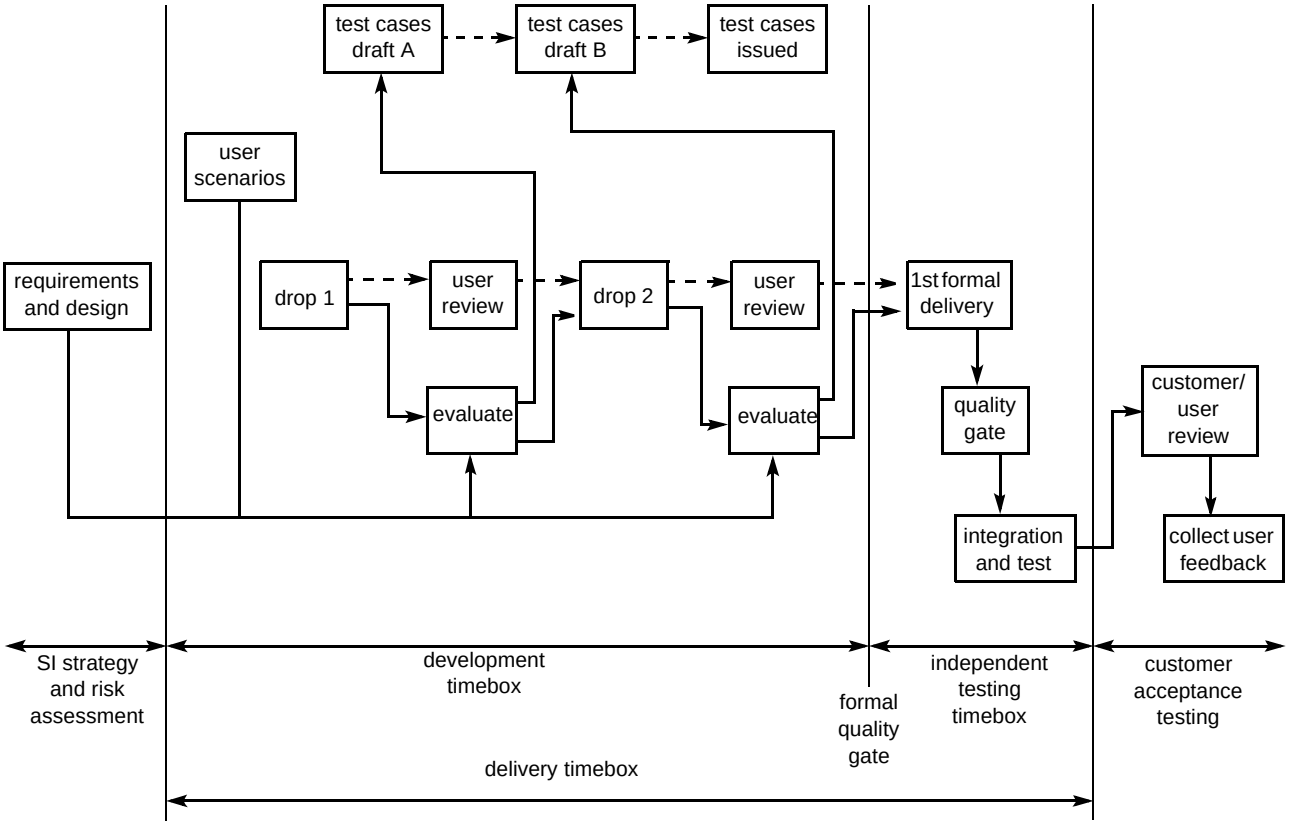

Fig 7 A more detailed depiction of the ‘**SI Rapid**’ process.

Standard testing techniques can be applied at this stage. These are fully documented in BT's VV&T handbook [1] and will not be discussed further here.

The more detailed view in Fig 7 shows the overall '**SI-Rapid**' testing process. The use of repeat drops within the timebox, and also the distinction between the development stage and the independent testing stage should be noted, both of which lie within the overall delivery timebox.

4.4 Customer acceptance testing

The final stage in the '**SI-Rapid**' life cycle model is the customer acceptance testing (CAT) (see Fig 8).

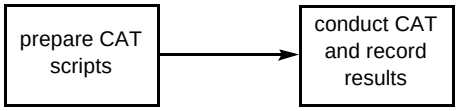

Fig 8 Customer acceptance testing.

The goal of CAT is to demonstrate to the customer, through the use of a small number of end-user scenarios, that the functionality being offered meets the needs of the customer. A prerequisite to this stage will be the availability of a customer acceptance test (CAT) script that may be produced by either the customer or the test team. If produced by the latter, a review and mutual sign-off will be required. It must be emphasised that the goal of the CAT is not to demonstrate that every functional element works correctly as this level of detail will be available from the test summary report.

During CAT, customer observations will need to be recorded so that these can be evaluated and, if agreed, become part of the requirements for the next release.

4.5 Challenges

There are a number of issues — problem reporting, test documentation, test coverage, quality, configuration management and maintenance — which are applicable to the entire '**SI-Rapid**' process and hence need special consideration.

Problem reporting

The type of problem reporting depends largely on where the project is in its life cycle. Within the early stages developers and testers can raise a problem report by simply writing the information on a note and sticking it on a problem report board. For small teams, in a localised development area, this is reported to work well. As the product matures the use of a formal tool, e.g. PCMS [6], is strongly recommended as such a tool offers a more reliable and consistent way of noting and tracking problems through the project.

Test documentation

Development often takes place informally with limited documentation. This is not helpful to the test team where as a minimum, workshop notes comprising customer requirements, installation and release notes and a user guide are required — and should accompany the software.

It may, however, prove difficult to get even this level of documentation from a development team. The '**SI-Rapid**' process aligns with DSDM by recognising the need to have documentation, but, whereas DSDM says a simple checklist is sufficient, '**SI-Rapid**' suggests that a more complete documented set, possibly based on generic SI templates, is required, comprising:

- test strategy,
- quality plan,
- test cases,
- test report.

Testing and test coverage

Due to tight time-scales in any DSDM development project, testing activities must be carefully prioritised. It is necessary to focus more carefully on those requirements and areas of functionality that are most important to the customer — the 20% of requirements that deliver 80% of the functionality. Prioritisation can be elicited from Customers by working through a test coverage/risk assessment matrix.

This approach is not without risk. Infrequently used or obscure items of functionality may only get a cursory look. The customer must be made aware of this and understand the level of risk involved.

Quality

It is important that the existing quality systems are not compromised by the '**SI-Rapid**' testing methodology. Nevertheless, it must be recognised that many of the standards laid down in traditional quality management systems will be difficult to meet. For this reason a project-specific quality plan should be produced to state what will be done and identify any concessions which may need to be agreed.

Configuration management

Close control of configuration management is seen as an essential element of '**SI-Rapid**'. While DSDM allows

testing on the development platform, experience shows that this can cause major problems for the test team as access to a stable test environment, when the development team want to keep moving forward, can create friction. Furthermore unless the development platform is identical to the deployment platform and a clean software build can be provided, the use of a separate independent test platform cannot be over-emphasised. The mere process of providing a build for installation on a 'clean' machine imposes a significant degree of configuration management, identifies problems which would otherwise not emerge until deployment and provides an excellent opportunity for installation note production.

Maintenance/maintainability

Maintainability is not a direct testing issue but it is very much a concern to the business and those responsible for in-life support — especially as those involved in the development may not be available later for support. For this reason alone, given the high cost of fault fixing on live systems, the extra effort of documentation and configuration management is easily justified.

4.6 Benefits

By taking elements of both the waterfall life cycle model and RAD as defined in DSDM, ' **SI-Rapid**' provides a sound but flexible environment for the delivery of small- to medium-scale projects. Test and development teams work in parallel as much as possible. Observations are noted quickly. Changes can be discussed and implemented promptly, thereby leading to the delivery of more robust code into formal test.

The high profile of the test team gives them a clear view of the product as it evolves, enabling them to produce better, more concise test cases. The team-working environment thus created allows all team members to concentrate their efforts on achieving project goals, and greatly increases the probability of first time success.

5. Making SI-Rapid work

5.1 Education

Without doubt the greatest challenge presented by ' **SI-Rapid**' is posed by the need to maintain contro l and discipline while moving quickly. Project management in such an environment is very demanding and requires that only high-calibre, skilled, and experienced project managers should be used. Rapid developments are not suited to 'first time' project managers. Similarly, all members of the project team need to be fully aware of their responsibilities and be given the authority to execute them.

Key individuals will require focused training in a range of tools and techniques. As a minimum, all should attend appropriate DSDM training courses. As ' **SI-Rapid**' combines features of traditional and DSDM approaches, a separate workshop will be necessary at the start of each project in order to define the process and provide a clear understanding of roles and responsibilities of all. It is essential that the customer attends this workshop. Rapid developments require rapid decisions and a much higher level of involvement in day-to-day operations during the life of the project than is often the case.

*5.2 '**SI-Rapid**' skill requirements*

'**SI-Rapid**' people need to have a broad base of VV&T, SI, organisational, team working and interpersonal skill. It is important that individuals are given the opportunity to build skills within the project framework and that time is allowed for formal training if required. As stated earlier, the role of the project manager is a particularly demanding one requiring a combination of experience, skills, enthusiasm and pragmatism. It is suggested that a minimum skill/experience set be defined as a necessary pre-condition for appointment to either overall project manager or systems integration team leader roles.

5.3 Virtual groups

The nature of BT's global business means that it will more often than not be very difficult to bring a team physically together to form a DSDM team. ' **SI-Rapid**' is not conditional on co-location with the development team by all team members but does require positive steps to be taken to ensure that a 'virtual' team can be formed across a frequently large geographical divide. Customers may also not be able to leave their normal place of work. Again this requires a positive solution to communication to be implemented. The increasing availability of video-conferencing suite resources has offered projects a way forwards and has proved very successful. Equally appropriate is the use of a PC-based videoconferencing system such as the BT VC8000, as this allows workplace to workplace communication without the formality of the videoconferencing suite, which normally has to be pre-booked for a set period. Although the virtual group would seem to be almost inevitable within BT, it is important that members of the various teams have the opportunity to 'bond' at the earliest opportunity. Whole team workshops followed by periods of secondment, e.g. the test team spend time with development during coding, then the development team spend time with the test team during formal test.

5.4 Effectiveness assessment

In the absence of so many other forms of documentation and records it might seem inappropriate to require that the teams collect metrics. However, this may need to be imposed because it is important that only the best combinations of techniques are used in a methodology that aims to reduce cycle times and costs. As a minimum, a post-implementation review should be held following each major deliverable so that there is the opportunity to deal

constructively with learning points and apply corrective action to points of issue.

6. Conclusions

Businesses are demanding ever-faster delivery of software solutions — especially in the areas of multi-media and business support. Major new developments and enhancement projects can continue to use the well-tried and trusted practices as defined in the VV&T and Systems Integration Handbooks because of their absolute need for integrity, accuracy, and maintainability.

Rapid application developments by their very nature require a lighter and more flexible approach, hence the introduction of DSDM. While many of the main features of DSDM can be retained in BT, some remain difficult to reconcile in an organisation as graphically spread and diverse as BT. As a consequence, an alternative, hybrid approach, known as '**SI-Rapid**' has evolved which combines the best features of traditional and DSDM methods to significantly improve the quality of a release without adversely affecting cost or project time-scales. In view of the flexibility of the '**SI-Rapid**' approach it is expected that the use of this method will become more widespread and become the standard for RAD-style systems integration and test.

'**SI-Rapid**' is not an easy option. The very nature of RAD is such that management control is difficult and the temptation to cut corners — especially on documentation and configuration management — is ever present. BT's Systems Integration department has a solid reputation for delivery built upon a long and successful track record. As the software industry approaches the millennium, the intelligent use of techniques such as '**SI-Rapid**' will continue to provide customers with the products they require — on time and to budget.

Acknowledgements

The authors would like to thank Steve Marsden of Systems Integration Department for his constructive and helpful feedback during preparation of this document; also, the DSDM Consortium who have a copyright interest in some of this material.

References

1 BT: 'VV&T Handbook, Issue 3', N&S Computing (1996).

2 BT: 'Systems Integration Handbook, Issue 3' N&S Systems Integration Department (1996).

3 BT: 'Systems Integration Process Handbook, Issue 2', (1996).

4 Dynamic Systems Development Method, version 2, DSDM Consortium, Published by Tesseract Publishing (December 1995).

5 Pryke W M et al: 'Systems integration throughout the early life cycle', BT Technol J, 15, No 3, pp9—18 . (July 1997).

6 PCMS, Software Ltd, Hertford.

Chris Carter joined BT as an apprentice in 1975. In 1981 he graduated from Essex University receiving a BSc (Hons) in Computer and Microprocessor Engineering and gained a Diploma in Management Studies from Suffolk College. In 1982 he joined BT Laboratories and from an initial software development career moved into systems integration. He has seven years experience in integration and test, which has included managing teams testing ServiceView and Service Creation. He is currently an integration manager leading teams testing RAD-developed applications for the account-based sales programme and multimedia customers.

Iain Whyte received a BSc (Hons) in Physics from the University of Southampton in 1974 and a Diploma in Management Studies from the Suffolk College in 1981. He joined BT in 1974 and worked on improving the reliability of transistors, integrated circuits, lasers and optical components used in subsea cable amplifiers.

He then moved on to work in software research looking at software requirements methods and measurement metrics. He currently leads a group in systems integration responsible for testing PSTN access and business systems. He is a Member of the IEE and a Chartered Engineer.

Stuart Birchall joined BT in 1963 as an apprentice in Liverpool undertaking duties in the transmission domain and trunk test. After graduating in 1972 he moved to BT Laboratories at Martlesham Heath to work on compiler testing and man machine interfaces associated with System X development. He went on to lead the contract with STC Ltd to develop a local administration centre (the forerunner of the OMC) for BT. Following this he moved to work on new generations of telecommunications systems, specialising in bringing together the technologies of telecommunications and computing. In recent years he has concentrated on leading the integration, testing and delivery of large, complex, computer-based systems employed throughout BT.

Dale Swatman joined Systems Integration at BT Laboratories after working for 5 years as a software engineer in the defence industry. He has been part of teams testing ServiceView and Service Creation.

He is currently leading a team within Systems Integration testing end-to-end functionality as part of the Cambridge Programme System Test activity.

He has a BSc (Hons) from Hatfield Polytechnic in Computer Science.